2026학년도 2학기 인문데이터 수량분석

제8강 텍스트 데이터

박수민

서울대학교 인문대학 연계전공 인문데이터과학

2026년 9월 30일 수요일

오늘의 목표

1. 문장을 단어로 쪼개고 품사와 기본형을 붙일 수 있다.
2. 그 문서에서만 두드러지는 단어를 골라낼 수 있다(TF-IDF).
3. 두 문서가 얼마나 가까운지 재고, 그 결과를 지도로 그릴 수 있다.
4. 영어가 아닌 언어의 텍스트에서 무엇이 달라지는지 말할 수 있다.

사전 준비

Posit Cloud의 qah-2026 프로젝트를 열고 콘솔에서 아래를 실행해 둔다.

```
install.packages(c("cleanNLP", "udpipe", "remotes"))
remotes::install_github(c("taylor-arnold/rpkg/ggimg",
                          "taylor-arnold/rpkg/hdir"))
```

⚠ hdir는 CRAN에 없다. CRAN판 ggimg로는 불러오기가 실패하니 둘을 함께 받는다.

1 **텍스트를 표로 바꾸기**
읽지 않고 읽는다는 것
오늘 쓸 자료

2 자연어 처리 파이프라인

3 문서의 특징을 보여주는 단어 골라내기

4 문서와 단어의 지도

5 다른 언어의 텍스트

6 마무리

수백 편을 다 읽을 수는 없다

연구자가 늘 마주치는 형편

조선왕조실록은 472년치 기록이 1,893권에 담겨 있다. 신문 한 종의 100년치, 문학 잡지 한 종의 전권도 마찬가지다. **한 사람이 평생 읽어도 다 못 읽는 분량**이 이미 디지털 텍스트로 있다.

그렇다고 안 읽을 수도 없다

자료가 무엇을 담고 있는지 모르면 질문조차 세울 수 없다.

오늘 배우는 타협

전부 세어 보고 **어디를 읽을지 정한다**. 계산은 읽기를 대신하지 않고 읽기를 안내한다.

➔ 이런 접근을 멀리서 읽기(distant reading)·문화 분석학(cultural analytics)이라 부른다.

❗ 세어 본 결과를 놓고 **다시 원문으로 돌아가는** 것까지가 한 세트다.

말뭉치·문서·토큰

세 단어만 익히면 된다

말뭉치 분석 대상이 되는 텍스트 전체. 영어로 corpus다.

문서 말뭉치를 나눈 한 덩이. **무엇을 한 문서로 볼지는 연구자가 정한다.**

토큰 문서를 더 쪼갠 단어 하나 또는 문장부호 하나.

문서의 단위가 분석을 바꾼다

시집 한 권을 한 문서로 볼 수도, 시 한 편을 한 문서로 볼 수도 있다. 잡은 단위에 따라 답이 달라진다.

오늘의 단위

위키백과 페이지 하나 = 문서 하나.
단어 하나 = 토큰 하나.

💡 6강에서 “관측 단위를 먼저 정하라”고 한 것과 같은 이야기다.

영국 작가 75명의 위키백과

```
docs <- read_csv(
  file.path("hdr", "data", "wiki_uk_authors_text.csv.bz2"))
docs
```

```
# A tibble: 75 × 2
  doc_id           text
  <chr>            <chr>
1 Marie de France  Marie de France was a poet possibly born in wh…
2 Geoffrey Chaucer Geoffrey Chaucer was an English poet author an…
3 John Gower       John Gower was an English poet a contemporary …
4 William Langland William Langland is the presumed author of a w…
# ℹ 71 more rows
```

➔ 한 행이 작가 한 명, `text` 열에 해당 작가의 문서 전체가 통째로 들어 있다.

메타데이터는 따로 둔다

```
meta <- read_csv(file.path("hdr", "data", "wiki_uk_meta.csv.gz"))
meta
```

```
# A tibble: 75 × 7
  doc_id             born  died era   gender link              short
  <chr>             <dbl> <dbl> <chr> <chr>  <chr>             <chr>
1 Marie de France    1160  1215 Early female Marie_de_France   Mari…
2 Geoffrey Chaucer   1343  1400 Early male   Geoffrey_Chaucer  Chau…
3 John Gower         1330  1408 Early male   John_Gower        Gower
# ℹ 72 more rows
```

➔ 생몰년·시대·성별·짧은 이름표. 4강에서 배운 `join`으로 언제든 이어 붙인다.

💡 텍스트와 메타데이터를 **한 표에 합치지 않은** 까닭은 크기다. 문서 전체를 담은 열을 join할 때마다 복사하면 자료가 걷잡을 수 없이 불어난다.

왜 하필 위키백과인가

실용적인 이유

- 누구나 받아 쓸 수 있다(오픈 액세스).
- 같은 주제가 300개 가까운 언어로 있다.
- 한 페이지 안에 글·표·링크·그림이 다 있다.

연구 대상으로서의 이유

- 무엇이 실릴 만한 지식으로 여겨지는가.
- 누구의 생애가 길게 적히는가.
- 같은 인물이 언어마다 어떻게 달리 그려지는가.

❗ 위키백과는 중립적인 거울이 아니라 **그 자체로 분석 대상**이다. 오늘 다루는 75명이 "영국 작가"를 대표한다고 볼 근거도 없다.

➔ 누가 목록에 들어왔고 누가 빠졌는지를 묻는 것부터가 연구다.

1 텍스트를 표로 바꾸기

2 **자연어 처리 파이프라인**
문서에서 토큰으로
표제어·품사·문장의 짜임

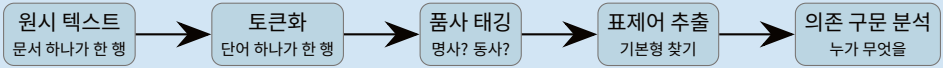
3 문서의 특징을 보여주는 단어 골라내기

4 문서와 단어의 지도

5 다른 언어의 텍스트

6 마무리

표의 모양을 한 번 더 바꾼다



파이프라인이 하는 일

문서 75행짜리 표를 받아 **토큰 40만 행짜리 표**를 내놓는다. 칸마다 언어학적 정보가 붙는다.

➔ 일단 표가 되면 그다음은 우리가 이미 아는 일이다.
`filter·group_by·summarize`로 다룬다.

💡 이 다섯 단계는 수십 년에 걸친 전산언어학 연구의 결과다. 우리는 갖다 쓰기만 한다.

cleanNLP와 udpipe

```
library(cleanNLP)

cnlp_init_udpipe("english")

anno <- cnlp_annotate(
  docs,
  doc_name = "doc_id",
  text_name = "text"
)$token
```

두 층으로 되어 있다

cleanNLP 어느 분석기를 쓰든 **같은 모양의 표**를 내주는 껍데기

udpipe 실제로 문장을 뜯어보는 알맹이

⚠ `cnlp_annotate`는 결과를 여러 표로 내놓는다. `$token`을 붙여야 우리가 쓸 표를 꺼낸다.

❗ 75페이지를 처리하는 데 몇 분이 걸린다. 오늘은 **미리 처리해 둔 결과**를 읽어 쓴다.

```
anno <- read_csv(file.path("hdr", "data", "wiki_uk_authors_anno.csv.gz"))
```

결과: 토큰 40만 행

```
# A tibble: 407,698 × 11
   doc_id    sid   tid token token_with_ws lemma upos  xpos  feats
   <chr>   <dbl> <dbl> <chr> <chr>         <chr> <chr> <chr> <chr>
 1 Marie …     1     1 Marie Marie         Marie PROPN NNP   Numb…
 2 Marie …     1     2 de    de            de    PROPN NNP   Numb…
 3 Marie …     1     3 Fran… France        Fran… PROPN NNP   Numb…
 4 Marie …     1     4 was   was           be    AUX   VBD   Mood…
 5 Marie …     1     5 a     a             a     DET   DT    Defi…
 6 Marie …     1     6 poet  poet          poet  NOUN  NN    Numb…
 7 Marie …     1     7 poss… possibly      poss… ADV   RB    <NA>
 8 Marie …     1     8 born  born          bear  VERB  VBN   Tens…
# ℹ 407,690 more rows
# ℹ 2 more variables: tid_source <dbl>, relation <chr>
```

➔ token 열을 위에서 아래로 읽으면 **원래 문장이 그대로 되살아난다.** 버린 것이 없다.

열 열한 개가 각각 무엇인가

어디에 있는 토큰인가

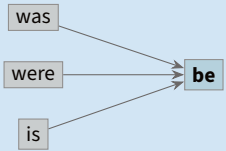
`doc_id`	어느 문서
`sid`	몇 번째 문장
`tid`	문장 안에서 몇 번째
`token`	본문에 적힌 그대로
`token_with_ws`	띄어쓰기까지 붙인 것

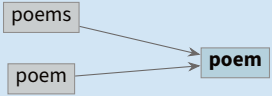
어떤 토큰인가

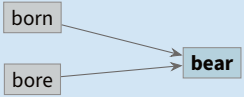
`lemma`	기본형(표제어)
`upos`	언어 공통 품사
`xpos`	그 언어만의 세부 품사
`feats`	시제·수 같은 문법 정보
`tid_source`	문법적으로 매달린 곳
`relation`	매달린 방식

💡 앞의 세 열 `doc_id`·`sid`·`tid`가 이 표의 기본키다. 4강에서 배운 그 기본키다.

표제어: 다른 꼴을 한 단어로 묶는다







왜 필요한가

poem과 poems를 따로 세면 둘 다 순위가 내려간다. 묶어 세야 “이 문서는 시를 많이 말한다”가 보인다.

➔ 그래서 대부분의 분석에서 token이 아니라 lemma를 센다.

⚠ 표제어는 기계가 짐작한 값이다. 늘 맞지는 않는다.

품사 태그 upos

자주 보게 될 것들

- **NOUN** 명사
- **PROPN** 고유명사
- **VERB** 동사
- **ADJ** 형용사
- **ADV** 부사
- **DET** 관형사·한정사
- **AUX** 조동사
- **PUNCT** 문장부호

"Marie de France was a poet"

Marie	was	a	poet
PROPN	AUX	DET	NOUN

쓰는 법

upos로 거르면 원하는 층만 남길 수 있다.
명사만 보면 **무엇을 말하는가**가, 동사만 보면 **무엇을 하는가**가 드러난다.

💡 upos는 언어가 달라도 같은 값을 쓴다. xpos는 언어마다 다르다.

한국어는 어떻게 쪼개지는가

```
cnlp_init_udpipe("korean")
cnlp_annotate(ko)$token
```

교착어의 한 어절

"김소월은"은 **김소월 + 은**이다.
뜻을 지닌 부분과 문법을 맡은 부분이 한 덩이로 붙어 있다.

	tid	token	lemma	upos	xpos
	<chr>	<chr>	<chr>	<chr>	<chr>
1	1	김소월은	김소월+은	NOUN	NNP+JX
2	2	진달래꽃을	진달래꽃+을	NOUN	NNG+JKO
3	3	썼다	쓰+었+다	VERB	VV+EP+EF
4	4	.	.	PUNCT	SF
5	1	그	그	DET	MM
6	2	시는	시+는	NOUN	NNG+JX
7	3	이별의	이별+의	NOUN	NNG+JKG
8	4	슬픔을	슬픔+을	NOUN	NNG+JKO
9	5	노래한	노래+하+ㄴ	VERB	NNG+XSV+ETM
10	6	작품이다	작품+이+다	VERB	NNG+VCP+EF

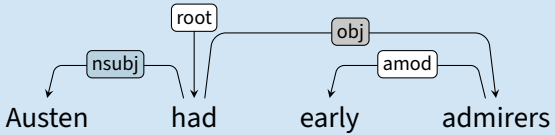
➔ 영어는 토큰 하나가 대체로 단어 하나지만, 한국어는 **한 토큰이 형태소 여럿**이다. `lemma`에 `+`로 이어 붙는다.

⚠ "작품이다"가 `VERB`로 붙었다. 서술격 조사 '이다' 때문이다. 분석기의 판단을 그대로 믿지 말고 **먼저 눈으로 확인**해야 한다.

의존 관계: 문장은 줄이 아니라 짜임이다

발상

문장은 단어를 늘어놓은 줄이 아니라, 각 단어가 **다른 단어 하나에 매달린** 짜임이다.



화살표 읽는 법

화살표는 **붙잡는 단어에서 매달린 단어로** 간다. `early`는 `admirers`에 매달렸다.

`root`란

아무 데도 매달리지 않는 **단 하나의 단어**.
대개 서술어이며, 여기서는 `had`다.

지어낸 문장이 아니라 위키백과 Jane Austen 문서의 253번째 문장이다.

그 짜임이 표에 적히는 방식

	tid	token	lemma	upos	tid_source	relation
	<dbl>	<chr>	<chr>	<chr>	<dbl>	<chr>
1	1	Austen	Austen	PROPN	2	nsubj
2	2	had	have	VERB	0	root
3	3	early	early	ADJ	4	amod
4	4	admirers	admirer	NOUN	2	obj
5	5	.	.	PUNCT	2	punct

한 행씩 읽으면

3행의 early는 **4번** 토큰(admirers)에 **amod**, 곧 형용사 수식으로 매달렸다.

tid_source가 0이면

매달린 데가 없다는 뜻, 곧 root다. 문장마다 반드시 하나 있다.

- ➔ 알아 둘 이름 셋: nsubj(주어 — 누가), obj(목적어 — 무엇을), amod(형용사 수식 — 어떤).
- ❗ 빈도만으로는 알 수 없는 것을 묻게 해 준다. “이 인물은 무엇을 **하는** 사람으로 그려지는가?”

작가를 주어로 삼는 동사 찾기

```
anno |>
  inner_join(meta, by = "doc_id") |>
  filter(short == lemma) |>
  filter(relation == "nsubj") |>
  select(doc_id, sid, tid_source) |>
  left_join(anno, by = c(
    "doc_id" = "doc_id",
    "sid" = "sid",
    "tid_source" = "tid")) |>
  count(lemma, sort = TRUE)
```

읽는 법

1. 작가 이름이 나온 자리를 찾고
2. 그 이름이 주어인 것만 남기고
3. 그 주어가 매달린 **동사**를 되찾아 온다.

➔ anno를 anno 자신과 잇는다. 4강의 join을 **같은 표에** 쓴 것이다.

그 결과

```
 [1] "write"    "have"     "begin"    "publish"  "become"
 [6] "leave"    "return"   "work"     "die"      "meet"
[11] "use"      "spend"    "make"     "receive"  "marry"
[16] "describe" "go"       "take"     "live"     "visit"
[21] "give"     "come"     "attend"   "believe"  "continue"
[26] "remain"   "argue"    "call"     "move"     "enjoy"
```

두 갈래가 보인다

창작에 관한 것(`write`, `publish`)과
생애에 관한 것(`marry`, `die`, `move`).

여기서 나오는 질문

남성 작가와 여성 작가에게 붙는 동사가 다른가? 수동태로 적히는 비율은 어떤가?

⚠ 이 방법은 `short` 이름이 그대로 주어로 나온 경우만 잡는다. 대명사로 받은 문장, 성이 같은 두 작가는 놓친다. **센 것이 무엇인지 정확히 알아야** 한다.

1 텍스트를 표로 바꾸기

2 자연어 처리 파이프라인

3 **문서의 특징을 보여주는 단어 골라내기**
단순 빈도의 한계
TF-IDF

4 문서와 단어의 지도

5 다른 언어의 텍스트

6 마무리

먼저 그냥 세어 본다

```
anno |>
  filter(upos == "NOUN") |>
  group_by(lemma) |>
  summarize(count = n()) |>
  arrange(desc(count))
```

```
# A tibble: 10,080 × 2
   lemma  count
   <chr>  <int>
 1 year    1008
 2 work    1001
 3 time     850
 4 life     753
 5 poem     741
 6 book     559
 7 death    539
 8 novel    508
 9 poet     489
10 family   475
```

두 갈래

생애에 관한 단어(year, life, death)과
작품에 관한 단어(poem, book, novel).

💡 품사로 먼저 거른 덕분에 the·of 같은 단어가 아예 올라오지 않는다.

그런데 이것으로는 작가를 구별할 수 없다

문제

`year·work·life`는 **75명 모두의 문서**에 많이 나온다. 모두가 공유하는 단어는 **누구도 설명하지 못한다.**

흔한 대처와 그 한계

불용어(stopword) 목록을 지우거나 품사로 거른다. 그러면 `publisher`처럼 **한 단계 덜 흔한 단어**가 그 자리를 차지할 뿐이다.

우리가 원하는 것

이 문서에는 자주 나오면서 다른 문서에는 잘 안 나오는 단어. 두 조건을 한꺼번에 따져야 한다.

➔ 7강에서 "단어마다 무게를 달리 주는 대각행렬"을 이야기했다. 그 무게를 정하는 방법이 오늘 나온다.

문서-단어 행렬을 다시 꺼낸다

	사랑	하늘	별	침묵	진달래
진달래꽃	40	22	3	1	18
님의 침묵	113	35	5	31	0
하늘과 바람과 별과 시	27	84	46	2	0

상황

시집 **여덟 권**을 모았고, 그중 세 권만 보인 것이라 하자. 수치는 실제로 센 값이 아니라 예시다.

➔ 단순 빈도로 줄을 세우면 세 권 모두 1등이 **사랑** 아니면 **하늘**이다.

❗ 그런데 우리가 알고 싶은 것은 **'님의 침묵'다운 단어**다.

두 개의 수를 곱한다

TF: 이 문서에 얼마나 자주 나오는가

tf는 그 단어가 이 문서에 나온 횟수다. 다만 **두 배 나왔다고 두 배 중요하지는 않으므로** 로그를 씌워 완만하게 만든다.

IDF: 얼마나 드문 단어인가

df는 그 단어가 한 번이라도 나온 **문서의 개수**다. N은 문서 전체의 개수다. 흔할수록 작아지도록 뒤집어 준다.

$$\mathrm{tfidf}(t,d) = \underbrace{(1+\log_2 \mathrm{tf}(t,d))}_{\text{이 문서에서의 무게}} \times \underbrace{\log_2 \frac{N}{\mathrm{df}(t)}}_{\text{드문 정도}}$$

모든 문서에 나온 단어는 $\mathrm{df} = N$이므로 $\log_2 1 = 0$, 곧 **점수가 0이 된다.**

계산 (1) 단어마다 드문 정도를 구한다

단어	df	$\log_2(8/\text{df})$	idf
사랑	8	$\log_2 1$	0
하늘	8	$\log_2 1$	0
별	4	$\log_2 2$	1
침묵	2	$\log_2 4$	2
진달래	1	$\log_2 8$	3

읽는 법

여덟 권 모두에 나온 **사랑**과 **하늘**은 무게가 **0**이다. 한 권에만 나온 **진달래**는 무게가 가장 크다.

여기서 중요한 것

idf는 **단어마다 하나씩**만 있다. 어느 문서를 보든 값이 같다. 말뭉치 전체가 정해 주는 수다.

그래서 말뭉치를 바꾸면 같은 단어의 무게가 달라진다. 시집만 모은 말뭉치에서 **사랑**은 흔하지만, 법전까지 섞으면 드물어진다.

계산 (2) 문서마다 곱해 본다

	진달래꽃	님의 침묵	하늘과 바람과
사랑	6.32×0	7.82×0	5.75×0
하늘	5.46×0	6.13×0	7.39×0
별	2.58×1	3.32×1	6.52×1
침묵	1.00×2	5.95×2	2.00×2
진달래	5.17×3	—	—

대표 단어	점수
진달래꽃 **진달래**	15.51
님의 침묵 **침묵**	11.91
하늘과 바람과 **별**	6.52

➔ **사랑**과 **하늘**은 아무리 많이 나와도 전부 0이 되어 사라졌다.

❗ 세 권에서 각각 제목에 해당하는 단어가 1등으로 올라왔다. **이것이 TF-IDF가 하는 일이다.**

R에서 한 줄로

```r
library(hdir)

anno |>
  filter(upos == "NOUN") |>
  hdir_text_tfidf(min_df = 0.05) |>    # 5% 미만 문서에 나오는 단어는 버린다
  arrange(doc_id, desc(tfidf))
```

```
# A tibble: 28,547 × 4
  doc_id       token           tf tfidf
  <chr>        <chr>        <int> <dbl>
1 A. A. Milne  toy              3 10.9
2 A. A. Milne  a                4  9.69
3 A. A. Milne  childrens        6  9.48
4 A. A. Milne  beneficiary      2  8.46
```

➔ `tf`(센 횟수)와 `tfidf`(무게를 준 점수)가 나란히 나온다.

⚠ 교재 97쪽에 적힌 식과 이 함수가 실제로 계산하는 식이 서로 다르다. 슬라이드는 **화면에 뜨는 수치를 내는 쪽**을 적었다.

문서마다 대표 단어 여덟 개

```
anno |>
  filter(upos == "NOUN") |>
  hdir_text_tfidf(min_df = 0.05) |>
  arrange(desc(tfidf)) |>
  group_by(doc_id) |>
  slice(1:8) |>
  summarize(token = paste(token, collapse = "; "))
```

```
1 A. A. Milne            toy; a; childrens; beneficiary; si…
2 Alexander Pope         pop; Pope; lock; Homer; Spectator;…
3 Alfred, Lord Tennyson  break; Isle; rectory; arm; asylum;…
4 Ann Radcliffe          terror; horror; landscape; Catholi…
5 Beatrix Potter         drawing; farm; farming; solicitor;…
6 Ben Jonson             comedy; tribe; folio; classicist; …
```

읽어 보면

맞힌 것

terror·horror·landscape는 고딕 소설가 Ann Radcliffe를 잘 집어냈다. drawing·farm은 그림을 그리며 농장에 살던 Beatrix Potter다.

뜻밖의 것

A. A. Milne의 목록에 toy가 1등으로 올랐다. 곰 인형 이야기를 쓴 작가이니 그럴 만하다.

걸리는 것

a, childrens, beneficiary처럼 오타·오분석이거나 한 번 스쳐 간 단어도 섞여 있다.

- tf가 너무 작은 것을 걸러 내면 한결 깨끗해진다. min_df 값을 바꿔 보는 것도 방법이다.
- 어느 값이 옳은지는 정해져 있지 않다. **여러 설정을 돌려 보고 결과를 비교하는** 것이 실제 작업이다.

1 텍스트를 표로 바꾸기

2 자연어 처리 파이프라인

3 문서의 특징을 보여주는 단어 골라내기

4 **문서와 단어의 지도**
- 거리 재기
- 차원 축소

5 다른 언어의 텍스트

6 마무리

단어 두 개만 남기면 평면에 그릴 수 있다

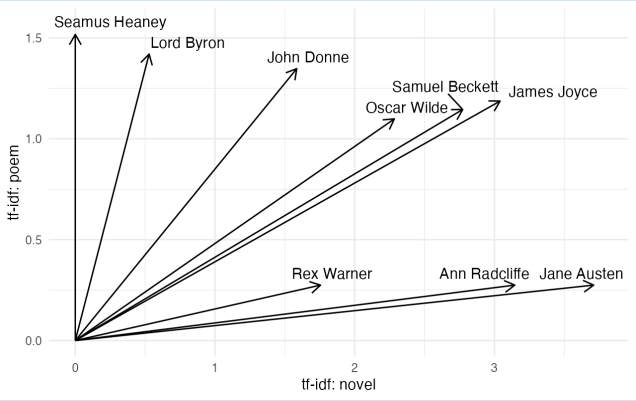
```
anno |>
  hdir_text_tfidf() |>
  filter(token %in%
    c("novel", "poem")) |>
  select(doc_id, token, tf) |>
  pivot_wider(
    names_from = "token",
    values_from = "tf",
    values_fill = 0)
```

```
# A tibble: 69 × 3
  doc_id                novel  poem
  <chr>                 <int> <int>
1 A. A. Milne               4     7
2 Alexander Pope            0    18
3 Alfred, Lord Tennyson     0    12
4 Ann Radcliffe            31     1
5 Ben Jonson                0    16
6 Bram Stoker              16     0
```

➔ 5강의 `pivot_wider`다. 긴 표가 **문서** × **단어** 행렬이 되었다.

⚠ 단어를 다 쓰면 열이 수만 개가 된다. `pivot_wider`로는 감당하지 못한다. 곧 다른 길을 쓴다.

아홉 명을 그려 보면



세 무리

- 오른쪽 아래: 소설가
- 왼쪽 위: 시인
- 가운데: 둘 다 쓴 사람

점이 아니라 화살표

원점에서 뻗은 화살표로 그렸다. 7강의 **벡터**다.

단어 둘이면 평면, 단어 1만 개면 1만 차원이다. **그릴 수 없을 뿐 계산은 똑같다.**

먼저 곧은 거리로 재 본다

```
anno |>
  cnlp_utils_tfidf(token_var = "lemma") |>    # 문서 x 단어 행렬을 곧바로
  hdir_tidy_distance()                        # 모든 문서쌍의 거리
```

	document1	document2	distance
	<chr>	<chr>	<dbl>
1	Rex Warner	William Langland	41.9
2	William Langland	Rex Warner	41.9
3	Felicia Hemans	Rex Warner	47.4
4	Edward Upward	Rex Warner	47.8
5	Katherine Philipps	William Langland	52.3

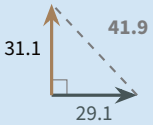
`cnlp_utils_tfidf`는 `pivot_wider` 없이 큰 행렬을 만든다. 빈칸이 대부분인 행렬을 위한 전용 도구다.

20세기 작가 Rex Warner와 14세기 시인 William Langland가 **가장 가까운 한 쌍**이라고 한다. 납득이 되는가?

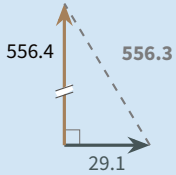
범인은 분량이다

세 문서의 크기와 사이각

Rex Warner 784, William Langland 971, Jane Austen 8,741 토큰. 벡터 길이는 차례로 29.1·31.1·556.4다. 세 쌍의 각도 거리는 모두 0.49, 곧 **거의 직각**이다.



Warner와 Langland
둘 다 짧다



Warner와 Austen
한쪽만 길다

➔ 직각이면 곧은 거리는 피타고라스 정리로 정해진다. 그림의 점선이 그것이다.

❗ 거리를 정하는 것은 어휘가 아니라 **긴 쪽의 길이**다. 해법은 7강에서 배웠다 — **방향만** 본다.

각도로 재면

```
anno |>
  cnlp_utils_tfidf(token_var = "lemma") |>
  hdir_tidy_angle_distance()
```

	document1	document2	distance
	<chr>	<chr>	<dbl>
1	George Herbert	Henry Vaughan	0.243
2	C. S. Lewis	Cecil Day-Lewis	0.281
3	Charlotte Brontë	Emily Brontë	0.287
4	Geoffrey Chaucer	John Gower	0.307
5	Louis MacNeice	W. H. Auden	0.314
6	James Boswell	Samuel Johnson	0.336
7	Ben Jonson	William Shakespeare	0.336

➔ 두 화살표가 벌어진 각을 $180°$로 나눈 수다. **0**이면 방향이 같고 **0.5**면 직각이다. 7강의 코사인 유사도를 각으로 바꾼 것이다.

이번에는 읽힌다

정말 가까운 사이

- 브론테 자매(Charlotte / Emily)
- 중세 영어의 두 시인(Chaucer / Gower)
- 엘리자베스 시대 극작가(Jonson / Shakespeare)

그러나 조심할 것

C. S. Lewis와 Cecil Day-Lewis가 2등이다. 두 사람은 아무 관계도 없고 **성이 같아서** 가까워졌을 뿐이다.

➔ 고유명사(PROPN)를 빼고 다시 계산하면 이 짝은 사라진다. 어떤 단어를 넣고 뺄지가 곧 **무엇을 유사성으로 볼지**를 정하는 일이다.

❗ 완벽한 설정은 없다. 설정을 바꿔 가며 **어떤 이야기가 보이는지** 살펴보는 편이 낫다.

목록 대신 그림이 필요하다

가까운 쌍 목록의 한계

75명이면 짝이 2,775개다. 한 줄씩 읽어서는 **전체의 모양**이 보이지 않는다.

하고 싶은 일

앞의 화살표 그림을 **단어 전부를 쓰고도** 그리는 것. 1만 차원을 2차원으로 줄여야 한다.

차원 축소란

문서마다 딸린 수만 개의 값을 **단 두 개**로 줄이되, 원래 가까웠던 문서가 새 자리에서도 가깝도록 만드는 일이다.

줄이면 반드시 무언가를 잃는다. 7강에서 본 대로 **되돌릴 수 없는** 변환이다. 그림은 자료의 전부가 아니라 납작해진 그림자다.

주성분 분석(PCA)

```
anno |>
  cnlp_utils_tfidf(
    token_var = "lemma") |>
  hdir_tidy_pca(n = 2)
```

하는 일

자료가 **가장 넓게 퍼진 방향**부터 차례로 골라 남긴다. 7강의 고유벡터가 여기서 쓰인다.

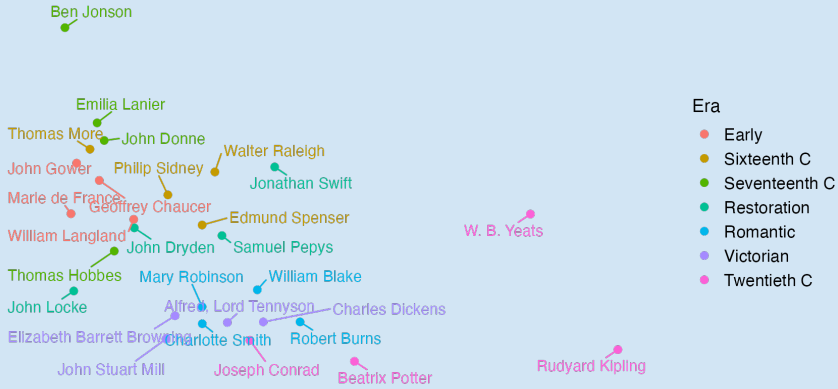
```
# A tibble: 75 × 3
  document              v1      v2
  <chr>              <dbl>   <dbl>
1 Marie de France  -0.0749 -0.120
2 Geoffrey Chaucer -0.0626 -0.163
3 John Gower       -0.0725 -0.186
4 William Langland -0.0476 -0.113
5 Margery Kempe    -0.0648 -0.100
```

v1과 v2

문서마다 좌표 두 개. 이제 ggplot으로 그릴 수 있다.

⚠ v1이 무엇을 뜻하는 축인지는 알려 주지 않는다. 단어 수만 개를 섞어 만든 값이다.

PCA로 그린 작가 지도



➔ 옛 작가들은 왼쪽에 빽빽이 몰려 있고 20세기 작가는 오른쪽으로 멀리 떨어졌다.

❗ 시대를 알려 준 적이 없는데 **단어 사용만으로** 갈라졌다. 시대마다 네 명씩만 그렸다.

UMAP

```
anno |>
  filter(upos %in%
    c("NOUN", "VERB", "ADJ", "ADV")) |>
  cnlp_utils_tfidf(token_var = "lemma") |>
  hdir_tidy_umap(n_neighbors = 3)
```

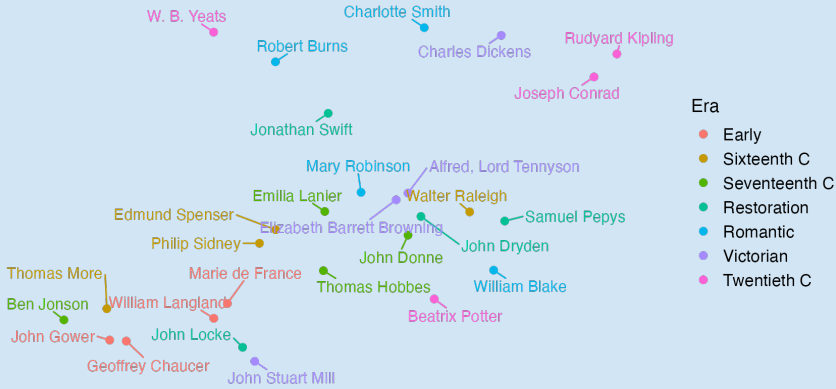
PCA와 다른 점

PCA는 **모든** 거리를 되도록 지키려 한다. UMAP은 **가까운 것들끼리의** 거리만 지키고 먼 것은 놓아 준다.

➔ n_neighbors가 작으면 잘게 나뉘고 크면 뭉뚱그려진다. 정답은 없고 몇 번 바꿔 봐야 한다.

⚠ UMAP에는 무작위성이 있어 **돌릴 때마다 그림이 달라진다.** `set.seed()`로 고정해 두어야 남이 같은 그림을 얻는다.

UMAP으로 그린 작가 지도



➔ 점이 고르게 퍼졌다. 초기·16세기 작가가 왼쪽 아래, 20세기 작가가 위쪽에 모였다.

❗ 어느 쪽이 "맞는" 그림인가? **둘 다 맞고 둘 다 일부만 보여 준다.**

행과 열을 맞바꾼다

```
library(Matrix)    # t()를 쓰려면 필요

anno |>
  filter(upos == "NOUN") |>
  cnlp_utils_tfidf(
    min_df = 0, max_df = 1,
    max_features = 400) |>
  t() |>                          # 전치
  hdir_tidy_angle_distance(
    item_name = "word")
```

관점이 바뀐다

문서가 행이면 **문서끼리** 비교하고,
단어가 행이면 **단어끼리** 비교한다.

가깝다는 말의 뜻

두 단어가 **같은 문서들에 비슷한 비율로** 나타난다는 뜻이다.

➔ 7강의 AA^T와 A^TA가 갈라지던 자리가 바로 여기다.

가장 가까운 단어 쌍

```
# A tibble: 399 × 3
  word1     word2       distance
  <chr>     <chr>          <dbl>
1 play      playwright     0.126
2 illness   room           0.140
3 time      year           0.142
4 evidence  scholar        0.149
5 poet      poetry         0.149
6 life      time           0.151
7 write     writing        0.154
8 novel     novelist       0.158
```

세 갈래

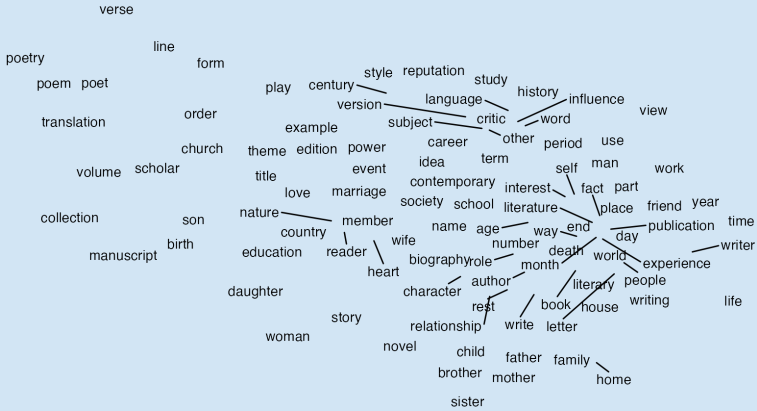
같은 뿌리 poet/poetry, novel/novelist

같은 주제 evidence/scholar — 연구가 많이 붙은 문서

같은 장면 illness/room — 투병과 임종을 적은 대목

💡 셋째 갈래가 특히 흥미롭다. 뜻이 닮아서가 아니라 **같은 이야기에 함께 나오기** 때문에 가깝다.

단어 지도



➔ 왼쪽에 시, 아래쪽에 가족, 오른쪽 위에 연구·비평의 단어가 모였다.

❗ 전치해 PCA에 넣었을 뿐인데 **주제별로 갈라진다.** 이름 붙이기는 연구자의 몫이다.

1 텍스트를 표로 바꾸기

2 자연어 처리 파이프라인

3 문서의 특징을 보여주는 단어 골라내기

4 문서와 단어의 지도

5 **다른 언어의 텍스트**

6 마무리

모델만 바꾸면 된다

```
docs_fr <- read_csv(
  file.path("hdr", "data", "wiki_uk_authors_text_fr.csv"))

cnlp_init_udpipe("french-partut")    # 프랑스어 모델로 갈아 끼운다
anno_fr <- cnlp_annotate(
  docs_fr, doc_name = "doc_id", text_name = "text")$token
```

나머지 코드는 그대로

upos와 relation의 값이 언어에 상관없이 같기 때문이다. **그러라고 만든 표준**이다.

한 언어에 모델이 여럿

프랑스어만 해도 partut·gsd·sequoia·spoken이 있다. 학습한 자료가 서로 다르다.

➔ 같은 인물의 위키백과 문서를 언어별로 받아 나란히 놓으면 **비교문화 연구**가 된다.

언어마다 생기는 잔가지

sid	tid	token	lemma	upos
<dbl>	<chr>	<chr>	<chr>	<chr>
1	4	est	être	AUX
1	5	une	un	DET
1	9	Rena…	Rena…	NOUN
1	10-11	du	<NA>	<NA>
1	10	de	de	ADP
1	11	le	le	DET

읽는 법

est는 être의 활용형, une는 un의 여성형으로 제대로 묶였다.

du가 세 줄을 차지한 까닭

프랑스어의 du는 전치사 de와 관사 le가 합쳐진 축약형이다. 원래 토큰 한 줄(tid가 10-11)과 풀어 쓴 두 줄이 함께 들어간다.

⚠ 이런 잔가지를 모르고 행을 세면 **토큰 수가 부풀어 오른다.** 새 언어를 다룰 때는 먼저 몇 문장을 눈으로 뜯어봐야 한다.

같은 사람, 다른 언어

```
1 A. A. Milne            bataille; attaque; champ; fut; Gue…
2 Alexander Pope         anglais; doute; tuberculose; andre…
3 Alfred, Lord Tennyson charge; second; écrire; ami; roi; …
4 Ann Radcliffe          roman; bourgeoisie; lorigine; cont…
5 Beatrix Potter         exemplaire; éditeur; conte; recher…
```

A. A. Milne — 영어판

toy, childrens, birthday, illustration
⇒ 아동문학 작가

A. A. Milne — 프랑스어판

bataille(전투), attaque(공격), champ(전장), guerre(전쟁)
⇒ 두 차례 세계대전의 참전자

- 같은 사람을 두고 두 언어 공동체가 **다른 이야기를 기억하고 있다.** 이것이 수치로 드러난 문화 차이다.
- → 프랑스어판이 영어판보다 훨씬 짧다는 사실도 함께 읽어야 한다.

한국어를 다룰 때

잘 되는 편

인도유럽어(영어·독일어·프랑스어·스페인어)와 셈어(아랍어·히브리어). 학습 자료가 많고 형태가 규칙적이다.

조심할 언어

띄어쓰기가 없는 중국어·일본어, 교착어인 한국어·튀르키예어, 학습 자료가 적은 언어.

한국어에서 특히 유의할 것

- 한 어절이 형태소 여럿으로 쪼개진다.
- 조사·어미가 붙은 꼴이 수없이 많아 표제어 추출이 필수다.
- 옛 표기·한자 혼용 문헌은 현대어로 학습한 모델이 잘 못 읽는다.

연구에 쓰기 전에 **자기 자료 100문장을 직접 채점해** 보아야 한다. 정확도를 모르는 채 낸 수치는 근거가 되지 못한다.

1 텍스트를 표로 바꾸기

2 자연어 처리 파이프라인

3 문서의 특징을 보여주는 단어 골라내기

4 문서와 단어의 지도

5 다른 언어의 텍스트

6 **마무리**

오늘의 함수

함수	하는 일	패키지
`cnlp_init_udpipe()`	언어 모델 준비	cleanNLP
`cnlp_annotate()`	문서 표 → 토큰 표	cleanNLP
`cnlp_utils_tfidf()`	문서 × 단어 행렬 만들기	cleanNLP
`hdir_text_tfidf()`	TF-IDF를 긴 표로	hdir
`hdir_tidy_distance()`	곧은 거리(유클리드)	hdir
`hdir_tidy_angle_distance()`	각도 거리	hdir
`hdir_tidy_pca()`	주성분으로 차원 축소	hdir
`hdir_tidy_umap()`	UMAP으로 차원 축소	hdir
`t()`	전치(문서 ↔ 단어)	Matrix

앞자리가 `cnlp_`면 cleanNLP, `hdir_`면 교재 저자들이 만든 패키지다.

오늘 배운 것

텍스트를 표로

- 말뭉치 $\rightarrow$ 문서 $\rightarrow$ 토큰
- 토큰마다 표제어·품사·의존 관계

그 문서다운 단어

- 단순 빈도는 흔한 단어만 보여 준다
- TF-IDF = 자주 나오는 정도 $\times$ 드문 정도
- 어디에나 나오는 단어는 0이 된다

문서 사이의 관계

- 곧은 거리는 분량에 휘둘린다
- 각도로 재야 어휘 구성이 보인다
- PCA·UMAP으로 지도를 그린다

단어와 언어

- 전치하면 단어끼리의 관계가 된다
- 모델을 갈아 끼우면 다른 언어도 된다

확인 질문

스스로 답해 보기

1. `token`과 `lemma`는 어떻게 다른가? 왜 대개 `lemma`를 세는가?
2. 모든 문서에 나오는 단어의 TF-IDF 점수가 0이 되는 까닭을 식으로 설명해 보라.
3. 말뭉치에 다른 종류의 문헌을 섞으면 어떤 단어의 idf가 달라지는가?
4. 곧은 거리로 잰 가장 가까운 문서 쌍이 엉뚱했던 까닭은 무엇인가?
5. C. S. Lewis와 Cecil Day-Lewis가 가까이 나온 것을 어떻게 손볼 수 있는가?
6. PCA로 그린 지도에서 가로축 `v1`이 무엇을 뜻하는지 알 수 있는가?

💡 답이 막히는 항목이 있으면 eTL Q&A 게시판에 질문을 올린다 (참여 점수 1점).

다음 시간

- 특강 (2026-10-07 수) — 인문데이터과학 특강
  - 2026-10-05 월요일은 개천절 대체공휴일이라 수업이 없다.
- **중간고사 (2026-10-12 월)** — 필기시험, 전체 성적의 30%
  - 범위는 1강부터 8강까지, 곧 교재 1–6장과 행렬 대수다.

시험까지 할 일

1. 2–6강에서 배운 동사(`filter`·`mutate`·`group_by`·`join`·`pivot`)를 손으로 써 보기
2. 숙제 네 개를 다시 열어 오류가 났던 대목을 확인하기

❗ 코드를 외우는 시험이 아니다. **왜 그 함수를 쓰는지** 설명할 수 있으면 된다.

더 보고 싶다면

- Universal Dependencies — https://universaldependencies.org/
- Silge & Robinson, *Text Mining with R* — https://www.tidytextmining.com/
- Jurafsky & Martin, *Speech and Language Processing* — https://web.stanford.edu/~jurafsky/slp3/
- 국립국어원 모두의 말뭉치 — https://corpus.korean.go.kr/
- 한국고전종합DB — https://db.itkc.or.kr/

교재의 안내

- 감정 분석·개념 추출·토픽 모형은 6장 마지막 절에 읽을거리가 있다.

➔ 토픽 모형(LDA)은 베이즈 통계가 필요해 교재 2판에서 빠졌다.